T **TechPlug POS**

TechPlug (Pvt) Ltd
Internal · Engineering

TECHNICAL HANDOVER

System architecture & development handover

Where everything lives, how the parts fit together, and how to keep building — for the TechPlug team taking over development.

React 19 + Vite + TypeScript | Offline-first · IndexedDB | Supabase (PostgreSQL) | Vercel | Tauri (Windows)

FOR
TechPlug developers and administrators

VERSION
1.0 · 14 November 2026

REPOSITORY
github.com/Matongos/restaurant-pos (main)

LIVE
pos.techplug.co.zw · Supabase project lrgeumaidmfmkszfdqjm (Frankfurt)

Confidential — internal

CONTENTS

What is in this document

1	Summary	What the system is, in one page
2	Where everything lives	Hosting, domains, accounts to hand over
3	Architecture	How the parts connect
4	Repository map	Which file does what
5	Data model	Device state and Supabase tables
6	How sync works	Outbox, pull, leader election, archiving
7	Main flows	Sale, online order, plans, payments, security
8	Working on it	Run, test, deploy, conventions
9	Known limits	Technical debt and suggested fixes
10	How to extend it	WhatsApp bot, Paynow, SMS and more
11	Runbooks	Common admin tasks, step by step

SECRETS

This document contains no passwords or secret keys, and none must ever be added to it or to the repository. The Supabase URL and *publishable* key in `web/src/cloudConfig.ts` are public by design; the `service_role`/secret key, database password and any provider API keys are not.

SUMMARY

1 The system in one page

TechPlug POS is a restaurant point-of-sale: till, kitchen screen, tables, change in several currencies, stock, books, staff PINs, a free public web page with QR menu and online orders per restaurant, branches, subscriptions, and an owner console for TechPlug.

It is **one React web app** that runs in the browser, installs as an app on phones and tablets (PWA) and is wrapped as a Windows app (Tauri). It works **offline first** — the whole restaurant lives on each device in IndexedDB — and syncs to **Supabase** (PostgreSQL) when online. There is **no server code of our own**: Vercel serves static files and all server logic is SQL (row-level security and `security definer` functions).

Front end
1 app
React 19 · Vite · TypeScript. Hash routes for till, kitchen, public pages and console.

Back end
SQL only
Supabase Auth + PostgREST + RPC functions. 4 migrations, all idempotent.

Hosting
Vercel
Push to `main` deploys in about a minute. Tills reload on a "new version" bar.

Area	Technology / decision
UI	React 19, plain CSS (`styles.css`), Lucide icons via `react-icons/lu`
State	One in-memory store (`store/store.ts`) persisted to IndexedDB; screens call store actions only
Sync	Outbox → `push_records()` ; pull every 8 s; last write wins per record
Auth	Supabase email + password per restaurant; staff use 4-digit PINs on the device; console admins need TOTP (aal2)
Public pages	`pos.techplug.co.zw/<slug>` → `public_menu()` , `place_online_order()` — customers never read restaurant data
Desktop	Tauri shell (`web/src-tauri`) that opens the hosted app; built by GitHub Actions on tags
Tests	Playwright browser tests against a Supabase stand-in (`tests/cloud-mock.cjs`); SQL tests on plain PostgreSQL

WHERE EVERYTHING LIVES

2 Hosting, domains and accounts

What	Where	Notes
Source code	GitHub `Matongos/restaurant-pos`, branch `main`	Everything below is in this one repository
Web app	https://pos.techplug.co.zw (Vercel project `web`)	Also web-delta-five-slu2k38rxm.vercel.app — the Windows app and early tills use it; same deployment
Product website	pos.techplug.co.zw/site/	Shown to first-time visitors of the bare address. Source `web/public/site/`
Owner console	pos.techplug.co.zw/console	TechPlug admins only: password + authenticator code
Restaurant pages	pos.techplug.co.zw/<slug> (`?t=T4` = table)	Rewritten by `web/vercel.json`; `main.tsx` turns it into `#/r/<slug>`
Database & logins	Supabase `lrgeumaidmfmkszfdqjm`, eu-central-1	URL + publishable key in `web/src/cloudConfig.ts` and `site.js`
DNS	cPanel (njiwacloud), zone techplug.co.zw	CNAME `pos` → Vercel
Windows installer	GitHub Releases	`.github/workflows/desktop.yml` on tags `desktop-v*`
Email	Not set up yet	Plan: Resend SMTP in Supabase — see `supabase/README.md`

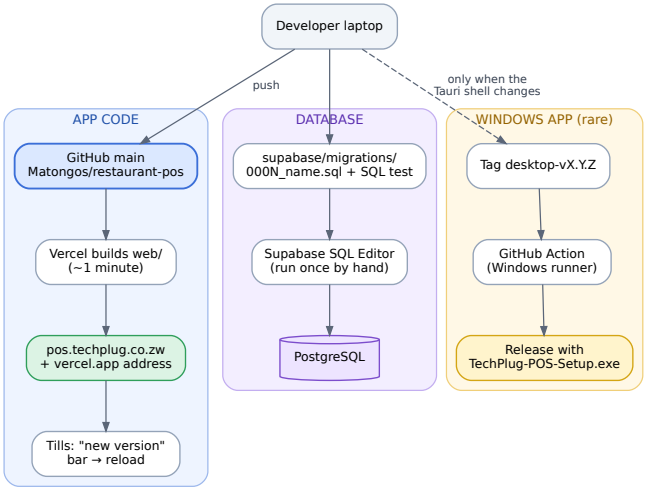


Figure 1. *From a commit to the tills: GitHub → Vercel for the app, SQL Editor for database changes, GitHub Actions for the Windows installer.*

ACCOUNTS TO HAND OVER

GitHub repo admin · Vercel team/project · Supabase organisation (owner) · cPanel for techplug.co.zw · console admin login (row in `platform_admins`) · later Resend and any payment/SMS provider. Turn on two-step login on every one of them.

HOW THE PARTS CONNECT

3 Architecture

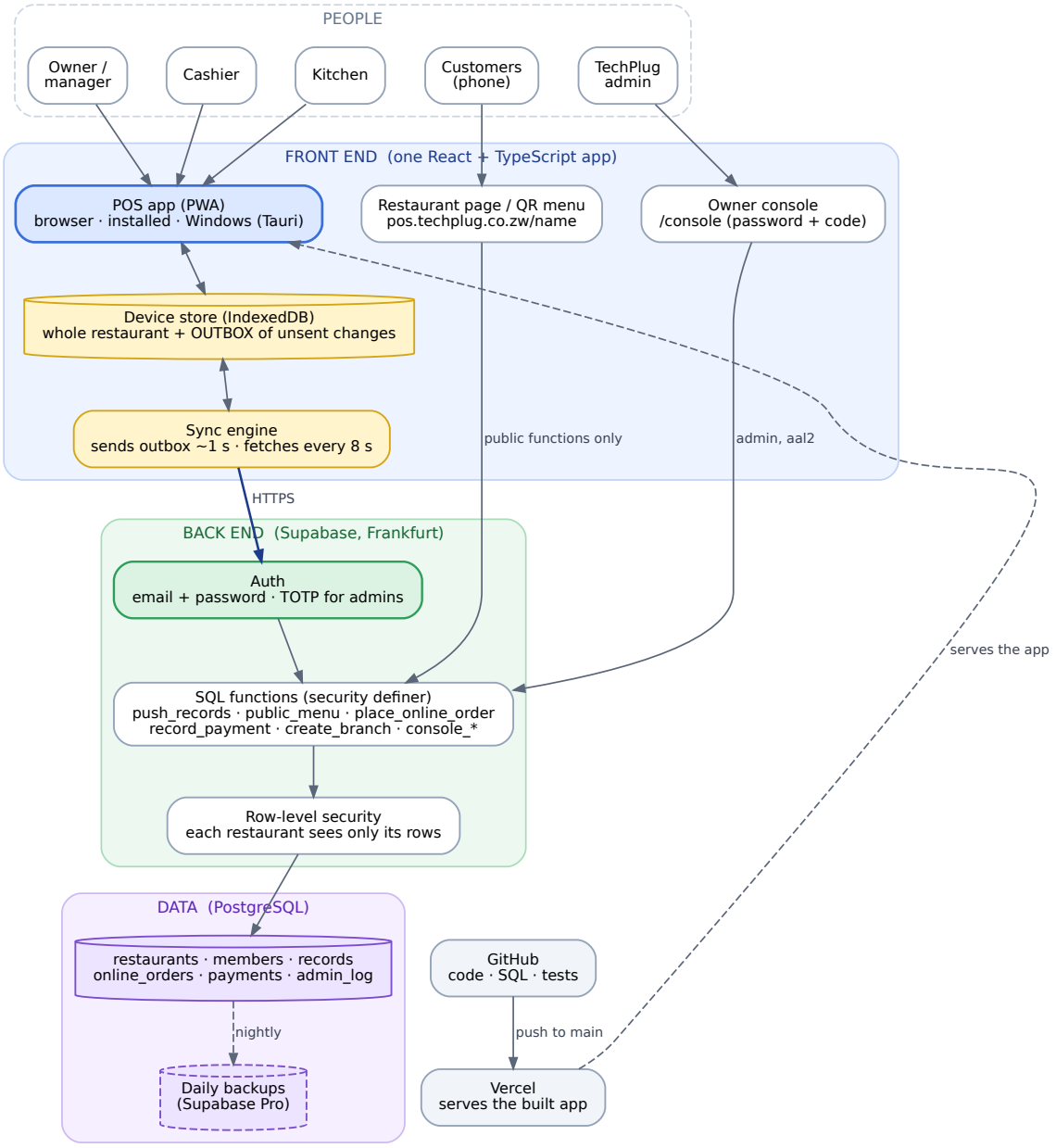


Figure 2. *System architecture. Blue: front end (one React app). Green: back end (Supabase). Purple: data. Dashed: build and backup paths.*

- **No server code of our own.** Vercel serves static files. All server logic is SQL in `supabase/migrations/*.sql`: tables, row-level security (RLS) and `security definer` functions.
- **Offline first.** The whole restaurant state lives in memory and in IndexedDB on each device (`web/src/store/disk.ts`). Selling never waits for the network.
- **One app, many screens.** Hash routes (`#/till`, `#/kitchen`, `#/r/<slug>`, `#/console` …) in `web/src/App.tsx`.
- **Customers never read restaurant data.** They only call `public_menu()`, `place_online_order()`, `online_order_status()` and `cancel_online_order()`. Prices are re-computed on the server.

THE CODE

4 Repository map

Path	What is in it
`web/src/main.tsx`	Start-up: website redirect for first visitors, short links, PWA, `initStore()`, sync, render
`web/src/App.tsx`	Routes, permissions per route, login gate, paused-till screen
`web/src/store/store.ts`	**All state and actions** (`placeOrder`, `cancelOrder`, `acceptOnlineOrder`, staff, stock…). Screens only call these
`web/src/store/disk.ts`	IndexedDB persistence, write queue, cross-window updates (BroadcastChannel)
`web/src/store/pinGuard.ts`	PIN lockout after wrong tries (5 free, then 30 s doubling to 15 min)
`web/src/domain/`	Pure rules: `types.ts` (data model), `rules.ts` (roles, permissions), `money.ts`, `stock.ts`, `tables.ts`, `analytics.ts`, `pinhash.ts`
`web/src/cloud/api.ts`	Fetch client for Supabase Auth/PostgREST (no SDK): sign-up/in, refresh, MFA, rpc/select/patch
`web/src/cloud/sync.ts`	Device ↔ cloud sync: outbox, pull, link, download, archiving, branch copy
`web/src/cloud/online.ts`	Restaurant info (plan, trial, paid-until), public menu publishing, online orders, sales lock
`web/src/cloud/platform.ts`	TechPlug details (WhatsApp, how to pay, prices) from `platform_settings`
`web/src/screens/`	One file per screen (Till = Sell, Analytics = Reports, Accounts = Books, Console, PublicPages, Setup…)
`web/src/components/`	Dialogs and settings panels (CloudAccount, WebsiteSettings, PlanPayments, MyAccount, Backup…)
`web/src/print/tickets.ts`	Slip, kitchen ticket and bill layouts
`web/src/data/seed.ts`	Sample restaurant (menu, staff, stock); `exampleHistory.ts` = example sales for training
`web/public/site/`	Product website (static HTML/CSS/JS, WhatsApp button reads `platform_settings`)
`web/public/sw.js`, `pwa.ts`	Service worker (offline app shell) and install prompts
`web/src-tauri/`, `desktop-shell/`	Windows app (Tauri) — opens the hosted app
`web/vercel.json`	Build settings, cache headers, short-link rewrite
`supabase/migrations/`	0001 core · 0002 console/online · 0003 branches/delivery/trial · 0004 payments/security
`supabase/tests/`	SQL tests run on plain PostgreSQL with `stub_supabase.sql`
`tests/`	Playwright browser tests and `cloud-mock.cjs`, a stand-in for Supabase

DATA

5 Data model

On the device

`AppState` in `domain/types.ts` (schema version 11, migrations in `store.ts › migrate()`): lists `staff, currencies, paymentMethods, orderTypes, categories, components, menu, addOns, orders, customers, credit, stockItems, stockMoves, purchases, expenses, cancellations, staffLog` and single objects `settings, dayCounter`.

Rule: records are **never edited away**. A cancellation, a reversing credit entry or a stock move corrects a mistake, so totals can always be traced.

In Supabase

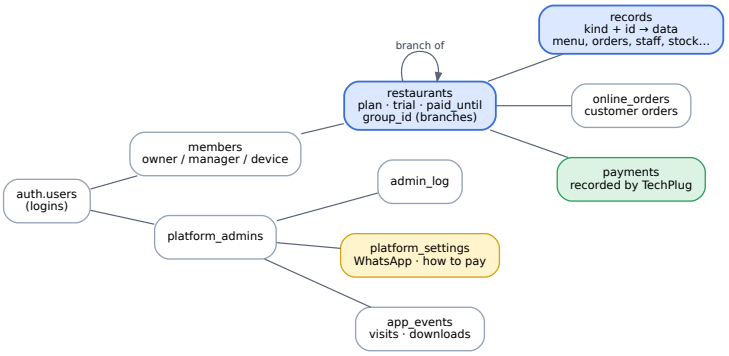


Figure 3. *Supabase tables and how they relate. Every device list item is one row in `records`.*

Table	Purpose	Who can read
`restaurants`	name, slug, plan, trial_ends_at, paid_until, group_id (branches), public switches, public_info, last_seen_at	members; owners update only public columns
`members`	login ↔ restaurant, role owner/manager/device	the login itself
`records`	`(restaurant_id, kind, id) → data jsonb`, deleted, device_id, server updated_at	members of that restaurant
`online_orders`	customer orders, server-priced; new → accepted → ready → out → collected	members; customers via token functions
`payments`	subscription payments recorded by TechPlug	owners of that group, admins
`platform_settings`	TechPlug WhatsApp, how to pay, prices	everyone read, admins write
`admin_log`	every admin action	admins
`app_events`	website visits, download clicks, first opens, sign-ups (throttled)	admins
`platform_admins`	TechPlug console logins	—

Key functions: `create_restaurant`, `slug_available`, `push_records`, `public_menu`, `place_online_order`, `online_order_status`, `cancel_online_order`, `create_branch`, `group_summary`, `restaurant_locked`, `record_payment`, `admin_set_plan`, `admin_set_platform`, `admin_delete_restaurant`, `admin_status`, `console_overview`, `console_restaurants`, `is_member`, `is_owner`, `is_platform_admin` (requires aal2).

OFFLINE FIRST

6 How sync works

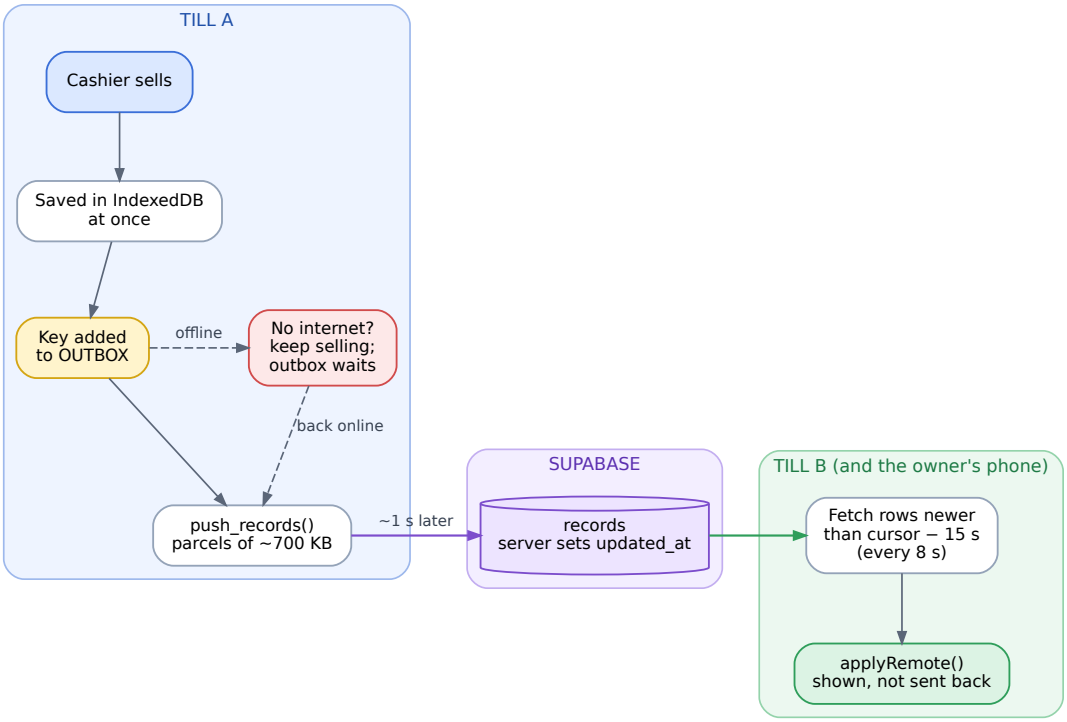


Figure 4. *Sync between a device and Supabase (`web/src/cloud/sync.ts`).*

1. Every local change goes through `set()` in the store. A change hook diffs old/new state **by object identity** and puts `kind\u0001id` keys in an **outbox** (kept in localStorage).
2. About a second later the leader window sends them with `push_records()` in parcels of ~700 KB.
3. Every 8 s (and when the app comes to the front) it fetches rows with `updated_at` newer than its cursor minus 15 s and applies them with `applyRemote()` (not re-sent). Last write wins; a change still in this device's outbox beats an older remote one; the device's own echoes are skipped.
4. Only one window per device syncs (Web Locks leader election); other windows get changes through IndexedDB + BroadcastChannel.
5. Each push doubles as a heartbeat (`last_seen_at`); the QR menu is "live" if a device was seen within 3 minutes.
6. A newly connected device downloads set-up data in full and the **last 6 months** of history (more if a stock item has not been counted since); "Load older sales" fetches the rest.
7. Slip numbers: each device has a till number (localStorage); till *n* uses numbers from (n−1)×500+1, so tills never collide offline.

FLOWS

7 Main flows

Sale

`Till.tsx` → `placeOrder()` (numbering, stock used, customer change, credit entries) → `OrderDoneDialog` / print → outbox → cloud.

Online order

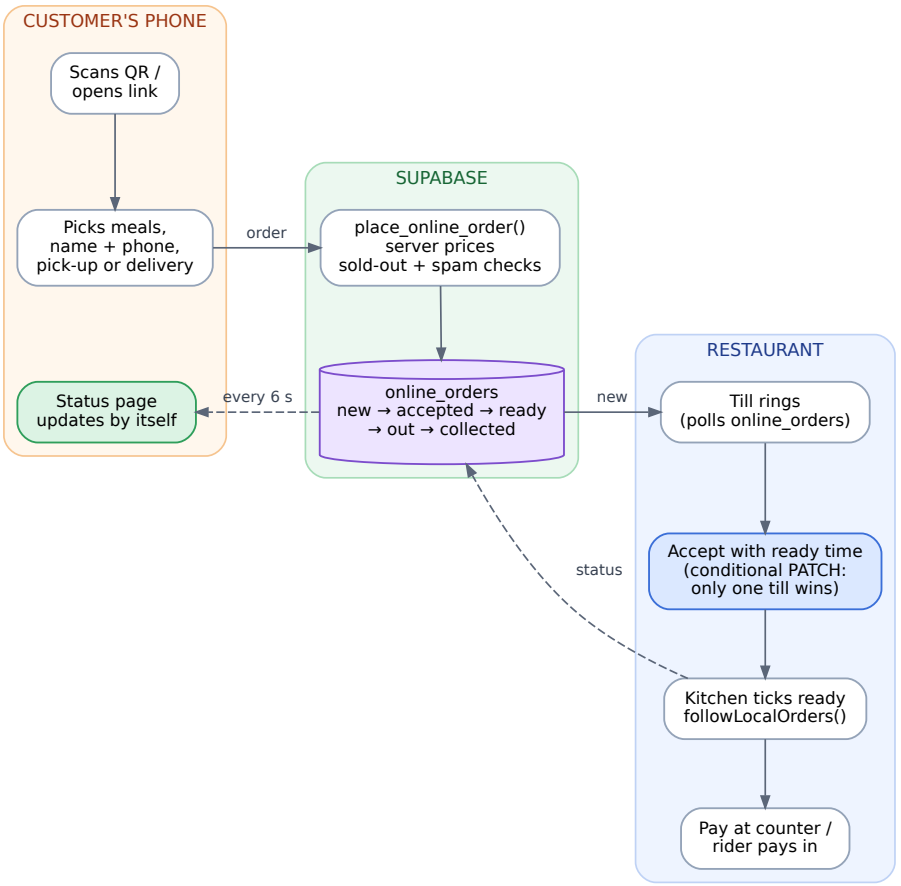


Figure 5. *An online order from the customer's phone to the till and back.*

Customer page `PublicPages.tsx` → `place_online_order()` (server prices, delivery fee, spam limits) → till polls `online_orders` → alert → `acceptRemote()` (conditional PATCH so only one till wins) → local order as a table tab `WEB n` → kitchen tick → `followLocalOrders()` sets ready/out/collected on the server. The public menu is built by the till (`publicMenuFrom` : sellable items, sold-out flags, delivery fee) and published as record `public/menu` when its hash changes.

Plans and payments

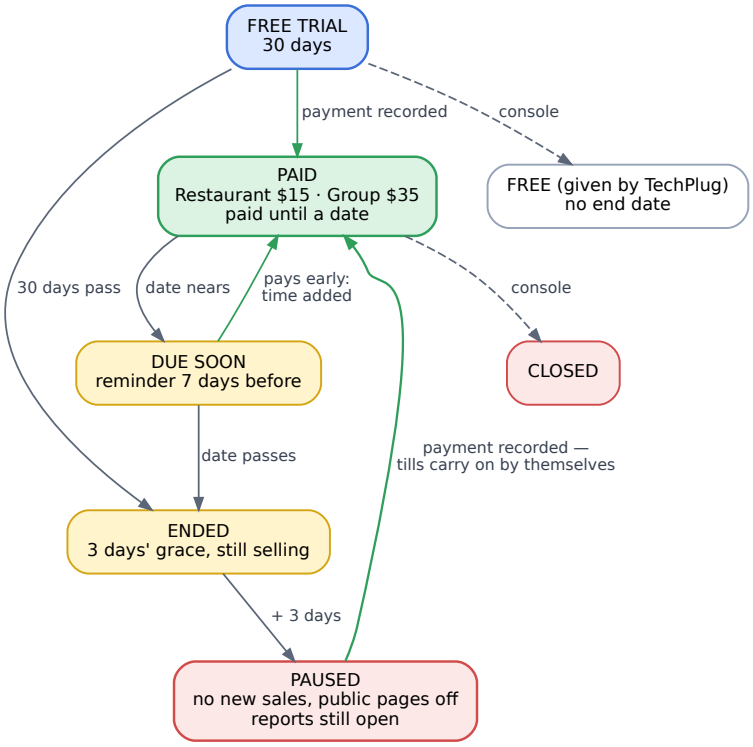


Figure 6. *Plan states.* `restaurant_locked()` *on the server and* `lockReason()` *on the till agree on the same rules.*

- **Lock:** `restaurant_locked()` stops public pages; `lockReason()` + `setSalesLock` block `placeOrder`, `sendTableRound`, `acceptOnlineOrder`. Grace = 3 days.
- **Payments:** console `record_payment()` → plan + `paid_until` for the whole group (extends from the current end or now) → tills re-read their info (every 5 min; every sync tick while locked) and unlock.

Security

Area	How it is protected
Console	Password sign-in, then TOTP enrol/verify (`/auth/v1/factors`) → JWT `aal2`. `is_platform_admin()` requires aal2. Idle sign-out after 30 min. Refuses to open on a linked till.
Staff PINs	Stored as `h1:` + SHA-256 stretched 3,000 rounds (`pinhash.ts`); plain PINs arriving from anywhere are scrambled (`scramblePins`). Lockout in `pinGuard.ts`.
Restaurant data	RLS: members read only their restaurant; owners update only public columns.
Public endpoints	Server-side prices; 3 open orders per connection per 10 min, 40 per restaurant per 10 min; app_events capped at 30/hour per connection.
Destructive admin actions	`admin_delete_restaurant` needs the exact name, deletes branches first, and is logged in `admin_log`.

DEVELOPMENT

8 Working on it

Run locally

```
cd web
npm install
npm run dev          # http://localhost:5173
npm run typecheck
npm run build
```

On `localhost` the app can talk to the Supabase stand-in instead of the real database: run `node tests/cloud-mock.cjs` (port 54321) and in the browser console `localStorage.setItem('restaurant-pos.cloud-url', 'http://localhost:54321')`.

Tests

- **Browser tests** (Playwright; app on `http://localhost:8765/` with the short-link rewrite, mock on 54321): `node tests/regression.cjs`, `offline`, `tables`, `staff`, `stock`, `sync`, `online`, `account`, `features`. Reset the mock between cloud tests with `GET /__reset`. Tests read data through `window.__pos` (localhost only).
- **SQL tests** on a scratch PostgreSQL: load `stub_supabase.sql` (skip its role line if roles exist), then `0001`, then `\i tests/test_console.sql`, `test_branches.sql`, `test_payments.sql` (they apply 0002–0004). `test_security.sql` is for 0001 alone.

Deploy

Change	How it goes live
App code	Push to `main` → Vercel builds `web` (`npm run build:deploy`) and deploys both addresses in ~1 minute. Tills show a "new version" bar and reload.
Database	Add `supabase/migrations/000N_<name>.sql` (idempotent: `create or replace`, `if not exists`), test it, then run it once in the Supabase SQL Editor.
Windows app	Tag `desktop-vX.Y.Z` → GitHub Action builds the installer and attaches it to a release. Only needed when the Tauri shell changes.

> **ORDER MATTERS**
> The app must keep working **before** a migration is run. See how `online.ts › refreshInfo` falls back when `group_id` or `paid_until` is missing, and do the same for new columns.

Conventions

- Screens call store actions only; business rules go in `domain/`, not in screens.
- Plain, short words in the UI, written for a busy cashier. No jargon.
- Records are permanent; corrections are new records.
- Every feature gets a browser test; every database change gets an SQL test.
- Small commits with a clear message, authored by the developer.

TECHNICAL DEBT

9 Known limits

Item	Impact	Suggested fix
Tills sign in with the owner's account	A stolen till's token has owner rights	Device logins (`members.role = 'device'` , anonymous sign-in + owner approval)
Plan lock only on connected tills	An unconnected till can sell for free	Require the online account after the trial
Last-write-wins per record	Two tills editing the same menu item at once: last wins	Fine for now; field-level merge if needed
Polling (8 s), not realtime	Up to 8 s delay between tills, more requests	Supabase Realtime on `records` / `online_orders` (already in the publication)
Browser print dialog	One extra click; no cash-drawer kick	ESC/POS printing in the Tauri app
Sample menu has no photos	Owners add their own	Add photos under `web/public/samples/` , reference in `seed.ts`
Email confirmation off	Unverified sign-up emails	Resend SMTP, then turn on Confirm email (wizard supports it)
Free Supabase plan	No backups	Supabase Pro ($25/month)
No "forgot owner PIN"	A single-owner restaurant needs TechPlug's help	PIN screen: account password → new owner PIN
Large `store.ts` (~1,800 lines)	Harder to navigate	Split by area keeping the same exports

ROADMAP

10 How to extend it

WhatsApp ordering bot

The bot reuses the exact path the web page already uses, so the till, kitchen and status updates work without changes.

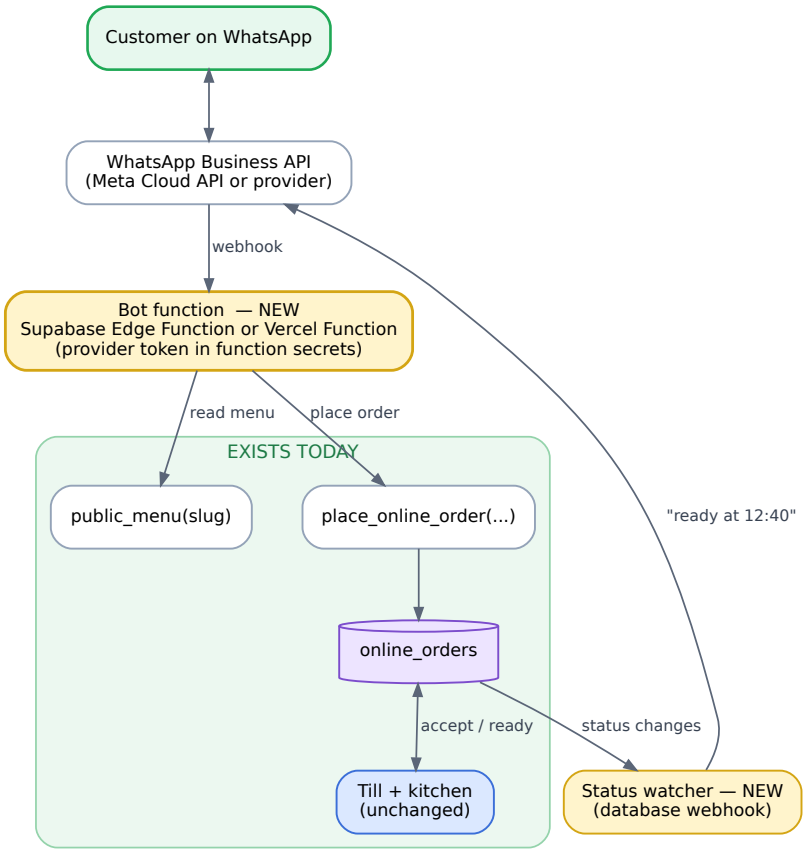


Figure 7. *Proposed WhatsApp bot. Only the webhook function is new; everything from `place_online_order()` onwards exists today.*

1. Get a WhatsApp Business API number (Meta Cloud API, or a provider such as Twilio / 360dialog).
2. Create a Supabase Edge Function (or Vercel Function) as the webhook. Keep the provider token in the function's secrets — never in the repo.
3. Map the incoming number to a restaurant (store the bot number in `restaurants.public_info`).
4. Read the menu with `public_menu(slug)`, build the cart in the chat (numbered list → quantities → name → pick-up/delivery).
5. Place the order with `place_online_order(...)` — same server pricing and spam limits.
6. Send "accepted / ready / on the way" messages by watching `online_orders.status` (database webhook or scheduled function).

Other ideas and where they plug in

Feature	How it would fit
Online subscription payments (Paynow: EcoCash/card)	Edge Function creates the Paynow transaction and receives its result URL; on success it calls the same logic as `record_payment()` (moved into an internal function called with the service role). Add "Pay now" in Settings → Plan & payments.
SMS when an order is ready	Same trigger as the WhatsApp status messages, through an SMS provider (e.g. Africa's Talking).
Device logins	See section 9.
Barcode scanning	USB scanners type the code + Enter: add `barcode` to `MenuItem` and a hidden input on Sell that matches it.
Loyalty	`customers` already exists (for kept change); add points to `credit` entries.
Languages (Shona, Ndebele)	Move UI strings into a dictionary per language; the text is already short and plain.
Owner notifications	No separate app needed: the PWA works; add push notifications through the service worker.
More reports / exports	`domain/analytics.ts` holds the calculations; add CSV export of Books and Reports.

OPERATIONS

11 Runbooks

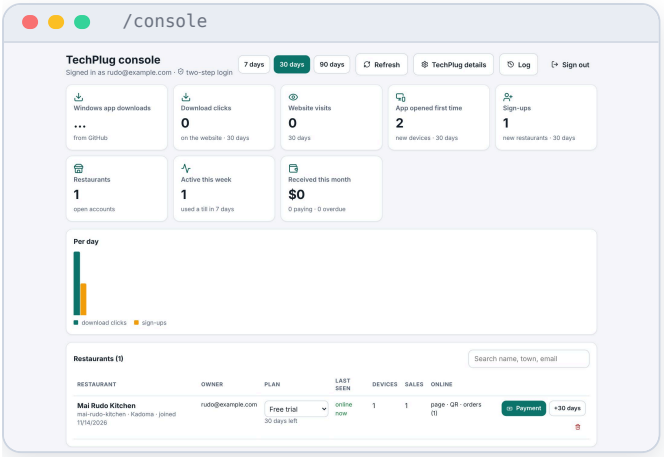


Figure 8. *Owner console: overview and restaurants.*

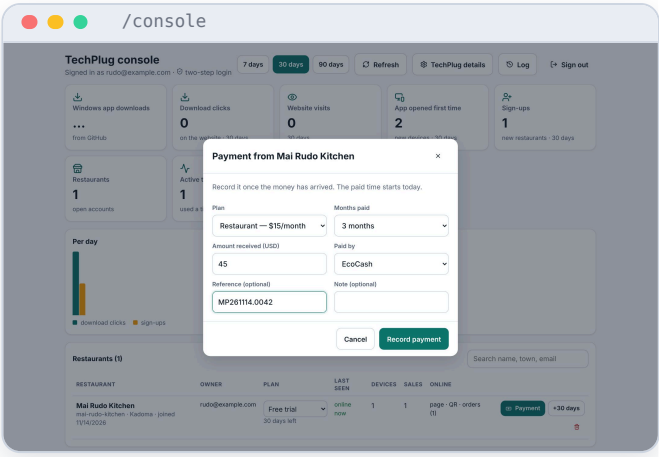


Figure 9. *Recording a payment.*

Make someone a console admin

```
insert into platform_admins (user_id)
select id from auth.users where email = 'name@example.com';
```

They set up their authenticator on first sign-in.

Reset a restaurant owner's password

```
update auth.users
set encrypted_password = extensions.crypt('NEW-PASSWORD', extensions.gen_salt('bf'))
where email = 'owner@example.com';
```

Clear the query afterwards. Their tills stay connected.

Task	Steps
Admin lost their phone	Supabase → Authentication → Users → the user → delete their MFA factor; they enrol again at next console sign-in.
Record a payment, give days, close or delete a restaurant	Owner console (all logged in `admin_log`).
Owner forgot their PIN	A second owner changes it under Staff. With a single owner there is no self-service reset yet (section 9).
Rotate keys	Supabase → Project Settings → API Keys. The publishable key is in `cloudConfig.ts` and `site.js`; rotating it needs a commit and deploy.
Restore a restaurant	Supabase Pro daily backups (point-in-time with the add-on); owners can also keep their backup file (Settings → Backup).

Other documents in the repository: `README.md` (features per screen) · `supabase/README.md` (Supabase set-up, email, security notes) · `docs/HANDOVER.md` (text version of this document) · `docs/ARCHITECTURE.md` (original design principles).